

Lecture 16

Gibbs sampling

Arnab Hazra

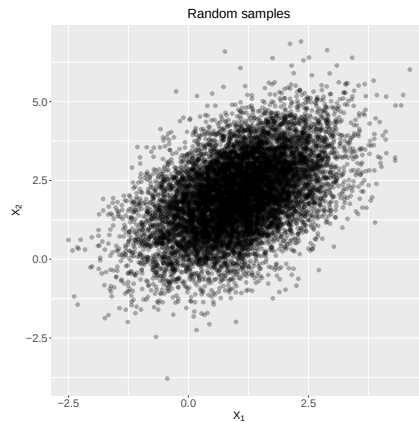
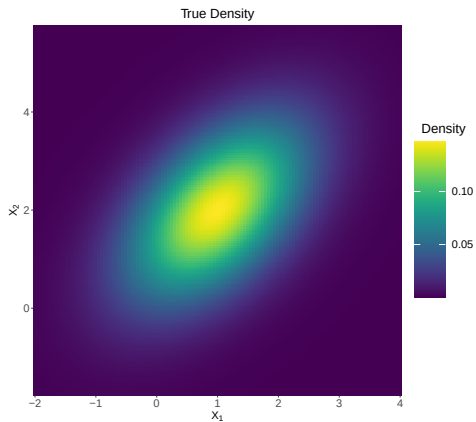
(Edited from the lecture notes by Brian J. Reich, NC State)



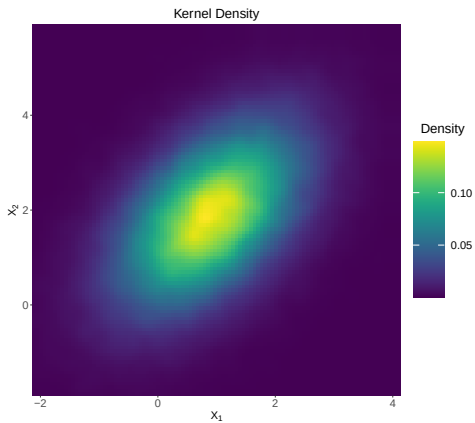
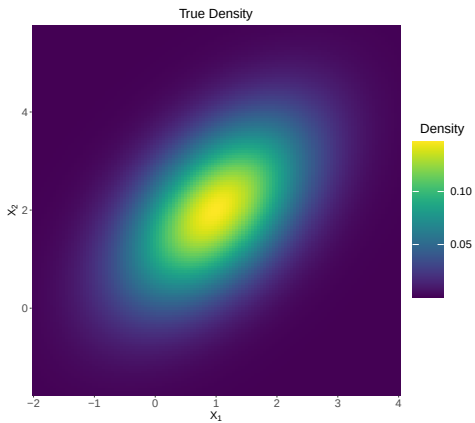
Monte Carlo sampling

- ▶ Monte Carlo (MC) sampling is the predominant method of Bayesian inference.
- ▶ The main reason is that it generally performs better than other alternatives for high-dimensional models (i.e., with many parameters).
- ▶ The main idea is to approximate posterior summaries in terms of some randomly drawn samples from the posterior distribution.
- ▶ This often requires drawing samples from non-standard distributions.
- ▶ It also requires careful analysis to be sure the approximation is sufficiently accurate.

Example



Example



Monte Carlo sampling

- ▶ Notation: Let $\theta = (\theta_1, \dots, \theta_p)'$ be the collection of all parameters in the model.
- ▶ Notation: Let $\mathbf{Y} = (Y_1, \dots, Y_n)'$ be entire dataset.
- ▶ The posterior $f(\theta|\mathbf{Y})$ is a distribution.
- ▶ If $\theta^{(1)}, \dots, \theta^{(S)}$ are samples from $f(\theta|\mathbf{Y})$, then the mean of the S samples approximates the posterior mean.
- ▶ This only provides approximations of the posterior summaries of interest.
- ▶ But how to draw samples from some arbitrary distribution $p(\theta|\mathbf{Y})$?

Software options

- ▶ There are now many software options for performing MC sampling.
- ▶ There are `R` functions for particular analyses (e.g., the function `BLR` for linear regression).
- ▶ There are also all-purpose programs that work for virtually any user-specified model: OpenBUGS; JAGS; Proc MCMC; STAN; INLA (not MC).
- ▶ We will use JAGS and also STAN (if possible), but they are all similar.

Markov chain Monte Carlo (MCMC)

We will study the algorithms behind these programs, which is important because it helps to

- ▶ Select models and priors,
- ▶ Anticipate bottlenecks,
- ▶ Understand error messages and output,
- ▶ Design your own sampler if these off-the-shelf programs are too slow.

The most common algorithms are **Gibbs** and **Metropolis** sampling.

Gibbs sampling

- ▶ Gibbs sampling is attractive because it can sample from high-dimensional posteriors.
- ▶ It breaks the problem of sampling from a high-dimensional joint distribution into a series of sampling from low-dimensional conditional distributions.
- ▶ Updates can also be done in blocks (groups of parameters).
- ▶ Because the low-dimensional updates are done in a loop, samples are not independent.
- ▶ The dependence turns out to be a Markov distribution, leading to the name Markov chain Monte Carlo (MCMC).

Gibbs sampling for the Gaussian model

- ▶ Say $Y_i \stackrel{iid}{\sim} \text{Normal}(\mu, \sigma^2)$ with $\mu \sim \text{Normal}(0, \sigma_0^2)$, and $\sigma^2 \sim \text{InvGamma}(a, b)$.
- ▶ In Chapter 2 (while discussing conjugate priors), we saw that if we knew either μ or σ^2 , we can sample from the other parameter.
- ▶ $\mu | \sigma^2, \mathbf{Y} \sim \text{Normal} \left(\frac{n\bar{Y}\sigma^{-2} + \mu_0\sigma_0^{-2}}{n\sigma^{-2} + \sigma_0^{-2}}, \frac{1}{n\sigma^{-2} + \sigma_0^{-2}} \right)$.
- ▶ $\sigma^2 | \mu, \mathbf{Y} \sim \text{Inverse-Gamma} \left(\frac{n}{2} + a, \frac{1}{2} \sum_{i=1}^n (Y_i - \mu)^2 + b \right)$.
- ▶ But how to draw from the joint distribution $\pi(\mu, \sigma^2 | \mathbf{Y})$?

Gibbs sampling for the Gaussian model

- ▶ The full conditional (FC) distribution is the distribution of one parameter taking all other as fixed and known.

- ▶ FC1: $\mu | \sigma^2, \mathbf{Y} \sim \text{Normal} \left(\frac{n\bar{Y}\sigma^{-2} + \mu_0\sigma_0^{-2}}{n\sigma^{-2} + \sigma_0^{-2}}, \frac{1}{n\sigma^{-2} + \sigma_0^{-2}} \right)$

- ▶ FC2: $\sigma^2 | \mu, \mathbf{Y} \sim \text{Inverse-Gamma} \left(\frac{n}{2} + a, \frac{1}{2} \sum_{i=1}^n (Y_i - \mu)^2 + b \right)$

Gibbs sampling

- ▶ In the Gaussian model $\theta = (\mu, \sigma^2)$ so $\theta_1 = \mu$ and $\theta_2 = \sigma^2$.
- ▶ The algorithm begins by setting initial values for all parameters, $\theta^{(0)} = (\theta_1^{(0)}, \dots, \theta_p^{(0)})$.
- ▶ Parameters are then sampled one at a time from their full conditional distributions,

$$p(\theta_j | \theta_1, \dots, \theta_{j-1}, \theta_{j+1}, \dots, \theta_p, \mathbf{Y}).$$

- ▶ Rather than 1 p -dimensional joint sample, we make p 1-dimensional samples.
- ▶ The process is repeated until the required number of samples have been generated.

Gibbs sampling

A Set initial value $\theta^{(0)} = (\theta_1^{(0)}, \dots, \theta_p^{(0)})$

B For iteration t ,

FC1 Draw $\theta_1^{(t)} | \theta_2^{(t-1)}, \dots, \theta_p^{(t-1)}, \mathbf{Y}$

FC2 Draw $\theta_2^{(t)} | \theta_1^{(t)}, \theta_3^{(t-1)}, \dots, \theta_p^{(t-1)}, \mathbf{Y}$

...

FCp Draw $\theta_p^{(t)} | \theta_1^{(t)}, \dots, \theta_{p-1}^{(t)}, \mathbf{Y}$

We repeat step B S times and get the posterior draws

$$\theta^{(1)}, \dots, \theta^{(S)}.$$

Why does this work?

- ▶ $\theta^{(0)}$ isn't a sample from the posterior and it is an arbitrarily chosen initial value.
- ▶ $\theta^{(1)}$ isn't likely from the posterior either. Its distribution depends on $\theta^{(0)}$.
- ▶ $\theta^{(2)}$ isn't likely from the posterior either. Its distribution depends on $\theta^{(0)}$ and $\theta^{(1)}$.
- ▶ **Theorem:** For any initial values, the chain will eventually converge to the posterior.
- ▶ **Theorem:** If $\theta^{(s)}$ is a sample from the posterior, then $\theta^{(s+1)}$ is too.

Convergence

- ▶ We need to decide:
 1. When has it converged?
 2. When have we taken enough samples to approximate the posterior?
- ▶ Once we decide the chain has converged at iteration T , we discard the first T samples as “burn-in”.
- ▶ We use the remaining $S - T$ to approximate the posterior.
- ▶ For example, the posterior mean (marginal over all other parameters) of θ_j is

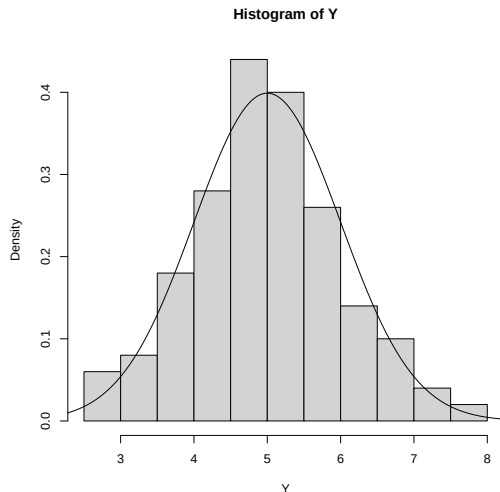
$$E(\theta_j | \mathbf{Y}) \approx \frac{1}{S - T} \sum_{s=T+1}^S \theta_j^{(s)}.$$

Let's code up (in R)

```
mu <- 5
sigmaSq <- 1

set.seed(100)
Y <- rnorm(100, mean = mu,
           sd = sqrt(sigmaSq))

hist(Y, probability = T)
grid.vals <- seq(mu - 4, mu + 4, 0.01)
y.vals <- dnorm(grid.vals, mean = mu,
                sd = sqrt(sigmaSq))
lines(grid.vals, y.vals)
```



Let's code up (in R)

- ▶ FC1: $\mu|\sigma^2, \mathbf{Y} \sim \text{Normal} \left(\frac{n\bar{Y}\sigma^{-2} + \mu_0\sigma_0^{-2}}{n\sigma^{-2} + \sigma_0^{-2}}, \frac{1}{n\sigma^{-2} + \sigma_0^{-2}} \right)$
- ▶ FC2: $\sigma^2|\mu, \mathbf{Y} \sim \text{Inverse-Gamma} \left(\frac{n}{2} + a, \frac{1}{2} \sum_{i=1}^n (Y_i - \mu)^2 + b \right)$

```
mu.update <- function(Y, sigmaSq, mu0, sigmaSq0){  
  mu.posvar.inv <- length(Y) / sigmaSq + 1 / sigmaSq0  
  mu.posvar <- 1 / mu.posvar.inv  
  mu.posmean <- (sum(Y) / sigmaSq + mu0 / sigmaSq0) / mu.posvar.inv  
  out <- mu.posmean + sqrt(mu.posvar) * rnorm(1)  
  out}  
  
sigmaSq.update <- function(Y, mu, a, b){  
  sigmaSq.inv <- rgamma(1, shape = a + length(Y) / 2,  
                        rate = b + sum((Y - mu)^2) / 2)  
  sigmaSq <- 1 / sigmaSq.inv  
  sigmaSq}
```

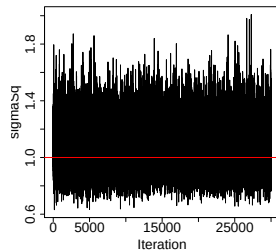
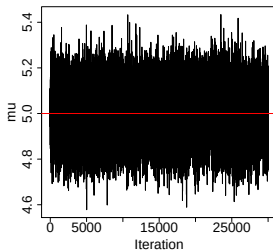
Let's code up (in R)

```
MCMC <- function(Y, mu.init, sigmaSq.init, mu0, sigmaSq0, a, b, iters){  
  
  # chain initiation  
  mu <- mu.init  
  sigmaSq <- sigmaSq.init  
  
  # define chains  
  mu.chain <- rep(NA, iters)  
  sigmaSq.chain <- rep(NA, iters)  
  
  # start MCMC  
  for(i in 1:iters){  
    mu <- mu.update(Y, sigmaSq, mu0, sigmaSq0)  
    sigmaSq <- sigmaSq.update(Y, mu, a, b)  
  
    mu.chain[i] <- mu  
    sigmaSq.chain[i] <- sigmaSq  
  }  
  
  # return chains  
  out <- list(mu.chain = mu.chain,  
              sigmaSq.chain = sigmaSq.chain)  
  return(out) }
```

Let's code up (in R)

```
MCMC.out <- MCMC(Y = Y,  
  mu.init = mean(Y),  
  sigmaSq.init = var(Y),  
  mu0 = 0,  
  sigmaSq0 = 100,  
  a = 0.1,  
  b = 0.1,  
  iters = 30000)
```

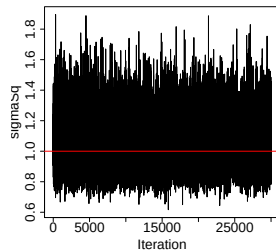
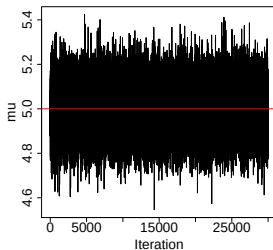
```
plot(MCMC.out$mu.chain, xlab = "Iteration", ylab = "mu", type = "l")  
plot(MCMC.out$sigmaSq.chain, xlab = "Iteration", ylab = "sigmaSq", type = "l")
```



Let's code up (in R)

```
MCMC.out <- MCMC(Y = Y,  
  mu.init = -10,  
  sigmaSq.init = 10,  
  mu0 = 0,  
  sigmaSq0 = 100,  
  a = 0.1,  
  b = 0.1,  
  iters = 30000)
```

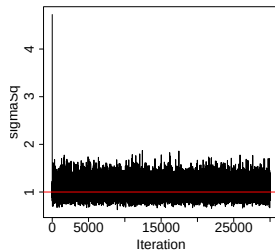
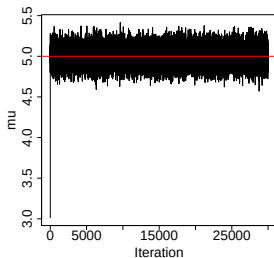
```
plot(MCMC.out$mu.chain, xlab = "Iteration", ylab = "mu", type = "l")  
plot(MCMC.out$sigmaSq.chain, xlab = "Iteration", ylab = "sigmaSq", type = "l")
```



Let's code up (in R)

```
MCMC.out <- MCMC(Y = Y,  
  mu.init = -1000,  
  sigmaSq.init = 1000,  
  mu0 = 0,  
  sigmaSq0 = 100,  
  a = 0.1,  
  b = 0.1,  
  iters = 30000)
```

```
plot(MCMC.out$mu.chain, xlab = "Iteration", ylab = "mu", type = "l")  
plot(MCMC.out$sigmaSq.chain, xlab = "Iteration", ylab = "sigmaSq", type = "l")
```



Another example

$Y|\lambda, b \sim \text{Poisson}(\lambda)$, $\lambda|b \sim \text{Exponential}(b)$, $b \sim \text{Exponential}(1)$.

- ▶ The full conditional for λ is

$$\begin{aligned} p(\lambda|b, Y) &\propto \frac{f(Y, \lambda, b)}{f(Y, b)} \propto f(Y, \lambda, b) \\ &\propto f(Y|\lambda, b)\pi(\lambda|b)\pi(b) \\ &\propto f(Y|\lambda)\pi(\lambda|b) \\ &\propto \exp[-\lambda]\lambda^Y \times b \exp[-b\lambda] \\ &\propto \lambda^{Y+1-1} \exp[-(1+b)\lambda]. \end{aligned}$$

- ▶ Therefore, $\lambda|b, Y \sim \text{Gamma}(Y+1, b+1)$.
- ▶ In the similar way, $b|\lambda, Y \sim \text{Gamma}(2, \lambda+1)$.

Thank you!